

Week 4 : Orders and Order Details Table

Scenario:

The E-commerce company sells product supplied by multiple sellers. The **Orders and Order Details tables** are important parts of an order management database. The ORDERS table stores the overall information about each customer order, while the ORDER_DETAILS table stores the individual products included in each order. These tables are connected using the ORDER_ID to maintain accurate and organized order information.

1. Create Orders and Order_Details Tables

Orders

```
SQL> CREATE TABLE Orders(Order_ID INT NOT NULL PRIMARY KEY, Customer_ID INT, Order_Date DATE, Total_Amount INT, Order_Status VARCHAR2(25));  
Table created.
```

Order_Details

```
SQL> CREATE TABLE Order_Details(Order_detail_ID INT, Order_ID INT NOT NULL, Product_ID INT NOT NULL, Quantity INT);  
Table created.
```

- Two tables for Orders and Order details Table have been created.
- As this table stores details of customer orders and their information.

2. Define Relationships and Constraints

Relationships:

The **Order** table stores information about the customer who orders ceramic products in the e-commerce system. Every order is stored in one table.

Order details table stores orders' essential details of each order.

- ❖ **One-to-Many Relationship:** One order in the ORDERS table can contain multiple products in the ORDER_DETAILS table.
- ❖ ORDER_ID connects the two tables, where one order can have **multiple order details/products**.
- ❖ This relationship is used to **find which products were included in each order and calculate purchase details such as quantity and subtotal**.

Constraints

- ✓ **PRIMARY KEY** – Uniquely identifies each record.
- ✓ **FOREIGN KEY** – Connects Product with Category.
- ✓ **NOT NULL** – Ensures a value must be entered.
- ✓ **CHECK** – Allows only valid values, such as Y or N.

1.we add primary key to Order details Table

```
SQL> ALTER TABLE Order_Details
  2  ADD CONSTRAINT PK_Order_Details PRIMARY KEY (Order_detail_ID);

Table altered.
```

2.we add check in order table for customization

```
SQL> ALTER TABLE Orders
  2  ADD CONSTRAINT chk_orders_customization
  3  CHECK (Customization_Avaliable IN ('YES','NO'));

Table altered.
```

3.we added foregin key product ID in order details table as both contain same values.

```
SQL> ALTER TABLE Order_Details
  2  ADD CONSTRAINT FK_Product
  3  FOREIGN KEY (Product_ID)
  4  REFERENCES Product(Product_ID);

Table altered.
```

We added new attribute in orders and order details table

```
SQL> ALTER TABLE Orders
  2  ADD Customer_Name VARCHAR2(25);

Table altered.

SQL> ALTER TABLE Order_Details
  2  ADD Customer_Name VARCHAR2(25);

Table altered.
```

```
SQL> ALTER TABLE Order_Details
  2  ADD Product_Name VARCHAR2(30);

Table altered.
```

4.we modified product id in order details table

```
SQL> ALTER TABLE Order_Details
  2  MODIFY Product_ID VARCHAR2(10);

Table altered.
```

5.we use INT as quantity will be stored in integer

```
SQL> ALTER TABLE Orders
  2  ADD Quantity INT;

Table altered.
```

6.we deleted customer name on both orders and order details table

```
SQL> DELETE FROM Order_details
  2  WHERE
  3
SQL> ALTER TABLE ORDER_DETAILS
  2  DROP COLUMN CUSTOMER_NAME;

Table altered.

SQL> ALTER TABLE ORDER_DETAILS
  2  DROP COLUMN PRODUCT_NAME;

Table altered.

SQL> ALTER TABLE ORDER_DETAILS
  2  ADD PRICE NUMBER(10,2);

Table altered.
```

Insert values in Order Table

```
SQL> INSERT INTO Orders
  2  (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status, Customization_Avaliable, Customer_Name, Product_Name, Quantity)
  3  VALUES
  4  (1001, 000101, DATE '2026-09-28', 550.00, 'Processing', 'YES', 'Sweety', 'Flower Vase', 2);

1 row created.

SQL> INSERT INTO Orders
  2  (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status, Customization_Avaliable, Customer_Name, Product_Name, Quantity)
  3  VALUES
  4  (1002, 000102, DATE '2026-08-27', 250.00, 'Dispatched', 'NO', 'Arun', 'Ceramic Bowl Set', 2);

1 row created.

SQL> INSERT INTO Orders
  2  (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status, Customization_Avaliable, Customer_Name, Product_Name, Quantity)
  3  VALUES
  4  (1003, 000103, DATE '2026-09-09', 600.00, 'Dispatched', 'YES', 'Priya', 'Coffee Mug', 4);

1 row created.

SQL> INSERT INTO Orders
  2  (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status, Customization_Avaliable, Customer_Name, Product_Name, Quantity)
  3  VALUES
  4  (1004, 000104, DATE '2026-09-15', 200.00, 'Dispatched', 'YES', 'Kumar', 'Ceramic Plate', 2);

1 row created.

SQL> INSERT INTO Orders
  2  (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status, Customization_Avaliable, Customer_Name, Product_Name, Quantity)
  3  VALUES
  4  (1005, 000105, DATE '2026-09-29', 1200.00, 'Processing', 'NO', 'Meena', 'Ceramic Tile Pack', 1);

1 row created.

SQL> INSERT INTO Orders
  2  (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status, Customization_Avaliable, Customer_Name, Product_Name, Quantity)
  3  VALUES
  4  (1006, 000106, DATE '2026-09-30', 750.00, 'Processing', 'YES', 'Ravi', 'Decorative Ceramic Figurine', 1);

1 row created.
```

Result

```
SQL> SELECT * FROM ORDERS;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS	CUS
1001	101	28-SEP-26	550	Processing	YES
Sweety		Flower Vase		2	
1002	102	27-AUG-26	250	Dispatched	NO
Arun		Ceramic Bowl Set		2	
1003	103	09-SEP-26	600	Dispatched	YES
Priya		Coffee Mug		4	
1004	104	15-SEP-26	200	Dispatched	YES
Kumar		Ceramic Plate		2	
1005	105	29-SEP-26	1200	Processing	NO
Meena		Ceramic Tile Pack		1	
1006	106	30-SEP-26	750	Processing	YES
Ravi		Decorative Ceramic Figurine		1	

6 rows selected.

Insert values in Order Details Table

```
SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID,Product_ID,Quantity ,Customization_Avaliable,Price)
  3 VALUES
  4 (1,1001,'P00004',2,'YES',550);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID,Product_ID,Quantity ,Customization_Avaliable,Price)
  3 VALUES
  4 (2, 1002,'P00005',4,'NO',250);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID,Product_ID,Quantity ,Customization_Avaliable,Price)
  3 VALUES
  4 (3,1003,'P00002',4,'YES',600);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID,Product_ID,Quantity ,Customization_Avaliable,Price)
  3 VALUES
  4 (4, 1004,'P00001',2,'YES',200);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID,Product_ID,Quantity ,Customization_Avaliable,Price)
  3 VALUES
  4 (5, 1005,'P00003',1,'NO',1200);

1 row created.

SQL> INSERT INTO Order_Details
  2 (Order_Detail_ID, Order_ID,Product_ID,Quantity ,Customization_Avaliable,Price)
  3 VALUES
  4 (6,1006,'P00003',1,'YES',750);

1 row created.
```

Result

```
SQL> SELECT * FROM ORDER_DETAILS;
```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	CUS	PRICE
1	1001	P00004	2	YES	550
2	1002	P00005	4	NO	250
3	1003	P00002	4	YES	600
4	1004	P00001	2	YES	200
5	1005	P00003	1	NO	1200
6	1006	P00003	1	YES	750

6 rows selected.

Report 1: To Display customer order history

- Displays customer name, order ID, order date, total amount, and status.
- Customer name is already available in the ORDERS table, so no CUSTOMER table is required.

```
SQL>
SQL> SELECT
  2  CUSTOMER_NAME,
  3  ORDER_DATE,
  4  TOTAL_AMOUNT,
  5  ORDER_STATUS
  6  FROM ORDERS
  7  ORDER BY CUSTOMER_NAME, ORDER_DATE;
```

CUSTOMER_NAME	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
Arun	27-AUG-26	250	Dispatched
Kumar	15-SEP-26	200	Dispatched
Meena	29-SEP-26	1200	Processing
Priya	09-SEP-26	600	Dispatched
Ravi	30-SEP-26	750	Processing
Sweety	28-SEP-26	550	Processing

6 rows selected.

REPORT 2: NUMBER OF TIME EACH PRODUCT ORDER

- Displays the number of times each product was ordered.
- Calculates the total quantity sold using **COUNT()** and **SUM()**.
- Uses **JOIN** to combine information from the customer, orders, and order-details tables.

```
SQL> SELECT
  2  P.PRODUCT_ID,
  3  P.PRODUCT_NAME,
  4  COUNT(OD.ORDER_ID) AS NUMBER_OF_TIMES_ORDERED,
  5  SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD
  6  FROM PRODUCT P
  7  JOIN ORDER_DETAILS OD
  8  ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 10  ORDER BY P.PRODUCT_ID;
```

PRODUCT_ID	PRODUCT_NAME	NUMBER_OF_TIMES_ORDERED	TOTAL_QUANTITY_SOLD
P00001	Dinner_Plate	1	2
P00002	Ceramic_Cup	1	4
P00003	Garden_Planter	2	2
P00004	Flower_Vase	1	2
P00005	Wall_Hanging	1	4

Report 3 : Customer Purchase Analysis

- Displays , order, product id, quantity details.
- combine information from the customer, orders, and order-details tables.

```
SQL> SELECT
  2     PRODUCT_ID,
  3     COUNT(ORDER_ID) AS NUMBER_OF_TIMES_ORDERED,
  4     SUM(QUANTITY) AS TOTAL_QUANTITY_SOLD
  5 FROM ORDER_DETAILS
  6 GROUP BY PRODUCT_ID
  7 ORDER BY PRODUCT_ID;
```

PRODUCT_ID	NUMBER_OF_TIMES_ORDERED	TOTAL_QUANTITY_SOLD
P00001	1	2
P00002	1	4
P00003	2	2
P00004	1	2
P00005	1	4

The relationship between ORDERS and ORDER_DETAILS helps manage customer orders efficiently. It allows us to identify the products in each order, their quantities, prices, and subtotals, making it easier to generate reports and analyze customer purchases.